

그리드 리뉴얼 — 단계별 구현 롤아웃

날짜: 2026-05-02 선행: 그리드 리뉴얼 — 자료구조 · encounter · 트리거 설계.pdf

목적: 그리드 리뉴얼 구현을 6 단계로 분할. 각 단계별 작업 범위 / 검증 시나리오 / 산출물 명시. 매 단계 헤드리스 자동 검증 후 다음 진입.

참조 원칙 (docs/범용적 원칙/): - 삼항 설계원칙.md — 상수 / 시물 / UI 3층 분리. 시물은 UI 무관해야. - 자료구조 상향식 구현 방법론.md — 자료구조 먼저, 함수는 그 위에 얹기. 리프부터. - 헤드리스 자동 검증.md — 단계별 자동 검증, 회귀 누적.

확정 결정 (선행 문서 §0 / §9): - 단일 슬롯 페어 (on_enter_slot + explore_slot) - 행동 비용 1 (지형 차 등 X) - 휴식 어디서나 + 랜덤 이벤트 - 강제 휴식 (actions=0) - 회귀 = 특정 노드 진입 (현 결) - 탐험 1 회차 1 회

§0. 어디서부터 — Pre-flight

0-1. 시작 지점

현 커밋: 5ec84fc (마스터, 세이브/로드 + 타이틀로 버튼). 회귀 127/127 PASS 상태.

0-2. 브랜치 전략

선택지: - (a) 마스터 직진 — 큰 커밋 N 개. 단순, 단 회귀 시 rollback 부담. - (b) grid-renewal 브랜치 — 안전. 단계별 커밋 후 일괄 merge.

→ (b) 권장. 6 단계 ÷ 일괄 PR 결로 안전 확보.

0-3. 자료구조 상향식 — 의존 DAG

```
WORLD_TERRAIN (const, leaf)
TERRAIN_RULES (const, leaf)
TILE_ENCOUNTERS (const, leaf — event_id 만 참조, EVENTS 풀과 분리)
EVENTS (const, leaf — 라이브러리)
    ↑
RunManager.run_data (player_pos / day / actions_remaining / visited / explored / ...)
    ↑
RunManager 함수 (try_move / try_explore / rest / encounter 평가)
    ↑
EventManager (combat kind 추가)
    ↑
run_ui (그리드 시각 + 입력)
    ↑
test/* (회귀 테스트 갱신)
```

리프부터 위로. 각 단계가 한 층씩 쌓아올림.

§1. 단계 표

단계	산출물	검증	의존
GR-1	자료구조 + 빈 시물 함수 시그니처	헤드리스 (자료구조 / 초기값)	—
GR-2	이동 + on_enter 트리거	헤드리스 (try_move / encounter 디스패치)	GR-1
GR-3	탐험 액션 + explore_slot	헤드리스 (try_explore)	GR-2
GR-4	휴식 / 일자 흐름 + 랜덤 이벤트	헤드리스 (rest / day++)	GR-2
GR-5	EventManager combat kind + research 진입 통합	헤드리스 (chain 안 전투 / 회귀)	GR-2
GR-6	UI 그리드 시각 + WASD + HUD	실 플레이 + 스크린샷	GR-2~5
GR-7	회귀 테스트 5 파일 갱신	회귀 127 → 갱신본 통과	GR-1~6
GR-8	세이브/로드 호환	헤드리스 (직렬화 / 복원)	GR-7

GR-6 까지가 시물 + UI 본체. GR-7~8 이 회귀 / 영속화 회복.

합 — 약 5~6일. GR-7~8 은 짧음 (체크 갱신만).

§2. Stage GR-1 — 자료구조 + 시물 함수 시그니처

작업 범위

`game_data.gd` :

```
const WORLD_TERRAIN = [
    "FFFFCCCCCCCCFFFF",    # 16×12 기준
    "FGGGGGGGGGGGGGGGF",
    ...
]

const TERRAIN_RULES = {
    "G": {"name": "풀밭", "passable": true},
    "P": {"name": "길",    "passable": true},
    "F": {"name": "숲",    "passable": false},
    "C": {"name": "절벽", "passable": false},
    "W": {"name": "물",    "passable": false},
}
```

```
const TILE_ENCOUNTERS = {} # 비어있음. GR-2 에서 채움.

const SPAWN_POS = Vector2i(1, 6) # 거점 위치
const ACTIONS_PER_DAY = 8
const DAY_MAX = 10
```

run_manager.gd : - `_new_big_run` / `_start_internal_run` 변경: - `current_node_id` / `map` 페기 - `player_pos` / `day` / `actions_remaining` / `visited_tiles` / `explored_tiles` / `seen_this_run` 신설 - 빈 함수 시그니처: - `try_move(dir: Vector2i) -> bool` — return false (스텝) - `try_explore() -> bool` — return false - `rest() -> void` — push_warning - `_get_terrain_at(pos: Vector2i) -> String` - `_is_passable(pos: Vector2i) -> bool`

검증 — test/test_grid_1.gd

- `GameManager.start_run()` 후 `RunManager.run_data` 의 새 필드 검증
- `player_pos == SPAWN_POS`
- `day == 1`, `actions_remaining == 8`
- `visited_tiles[SPAWN_POS] == true`
- `_get_terrain_at(SPAWN_POS)` 정상값 반환
- `_is_passable(Vector2i(0, 0)) = false` (벽)

체크 ~10 항목.

산출물

- 콘솔 로그 (`PASS` / `FAIL` 자동 판정)
- `git diff` — `game_data.gd` / `run_manager.gd`

§3. Stage GR-2 — 이동 + on_enter 트리거

작업 범위

run_manager.gd::try_move 본격 구현:

```
func try_move(dir: Vector2i) -> bool:
    if run_data.get("phase") == "event": return false
    var target: Vector2i = run_data["player_pos"] + dir
    if not _is_passable(target): return false
    if run_data["actions_remaining"] < 1: return false

    run_data["player_pos"] = target
    run_data["actions_remaining"] -= 1
    run_data["visited_tiles"][target] = true
```

```

_check_on_enter(target)

if run_data["actions_remaining"] <= 0:
    rest()

state_changed.emit()
return true

func _check_on_enter(pos: Vector2i) -> void:
    var slot = _get_on_enter_slot(pos)
    if slot.is_empty(): return
    if not _slot_passes_filters(slot): return

    seen_this_run[slot.id] = true
    match slot.kind:
        "event": _begin_event_phase(slot.event_id)
        "research": _enter_research()
        # combat / 그 외는 GR-5 에서

```

game_data.gd::TILE_ENCOUNTERS 채움:

```

const TILE_ENCOUNTERS = {
    Vector2i(8, 6): {
        "on_enter": {
            "id": "intro_at_spawn",
            "kind": "event",
            "event_id": "intro_speech",
            "once_per": "internal_run"
        }
    },
    Vector2i(12, 4): {
        "on_enter": {
            "id": "research_node",
            "kind": "research",
        }
    }
}

```

검증 — test/test_grid_2.gd

- `try_move(Vector2i(1, 0))` — 통과 가능 → 이동, actions=7
- `try_move(Vector2i(-1, -1))` — 벽 방향 → false, actions=7
- 이동 후 `visited_tiles[new_pos] = true`
- on_enter 슬롯 있는 타일 진입 → event 발화 (phase="event")
- once_per: internal_run 검증 — 같은 회차 재진입 시 미발화

체크 ~15 항목.

§4. Stage GR-3 — 탐험 액션 + explore_slot

작업 범위

```
func try_explore() -> bool:
    if run_data.get("phase") == "event": return false
    if run_data["actions_remaining"] < 1: return false

    var pos: Vector2i = run_data["player_pos"]
    if run_data["explored_tiles"].get(pos, false): return false

    run_data["explored_tiles"][pos] = true
    run_data["actions_remaining"] -= 1

    var slot = _get_explore_slot(pos)
    if not slot.is_empty() and _slot_passes_filters(slot):
        seen_this_run[slot.id] = true
        match slot.kind:
            "event": _begin_event_phase(slot.event_id)

    if run_data["actions_remaining"] <= 0:
        rest()

    state_changed.emit()
    return true
```

TILE_ENCOUNTERS 에 explore 슬롯 추가:

```
Vector2i(7, 5): {
    "explore": {
        "id": "find_letter_at_7_5",
        "kind": "event",
        "event_id": "find_letter",
        "once_per": "big_run"
    }
}
```

검증

- try_explore() 빈 타일 — false
- explore 슬롯 있는 타일 — event 발화
- 같은 회차 재탐험 — false

- 회귀 후 same big_run — once_per 따라 결정

체크 ~10 항목.

§5. Stage GR-4 — 휴식 / 일자 흐름

작업 범위

```
func rest() -> void:
    if run_data.get("phase") == "event": return

    # REST_EVENTS 추천
    var rest_event_id = EventManager.resolve_event("on_rest", {})
    if rest_event_id != "":
        _begin_event_phase(rest_event_id)
        # event 종료 시 _on_event_resolved 가 phase 복귀
        # 그 핸들러가 rest 후처리도 같이 처리하도록 분기
    else:
        _finalize_rest()

func _finalize_rest() -> void:
    run_data["day"] += 1
    run_data["actions_remaining"] = ACTIONS_PER_DAY
    run_data["explored_tiles"] = {} # 일자 reset? 정책 결정 - 회차 1 회로 같지
    state_changed.emit()
```

(주의) 탐험 reset 정책 — §1 결정상 "한 회차 1 회" — 일자 단위로 reset 안 함. explored_tiles 는 회차 끝까지 연속, end_internal_run 시 리셋.

game_data.gd::EVENTS 에 REST_EVENTS 풀 추가:

```
"rest_dream_1": {
    "id": "rest_dream_1",
    "kind": "dialogue",
    "trigger": {"type": "on_rest"},
    "weight": 5,
    "lines": [{"speaker": "히로인", "text": "밤이 — 짧다."}]
}
```

검증

- actions=0 도달 시 자동 rest 호출
- day +1, actions 리셋
- on_rest 트리거된 이벤트 발화

- explored_tiles 유지 (회차 1 회 정책)

체크 ~10 항목.

§6. Stage GR-5 — combat kind + research 진입

6-1. EventManager 의 combat kind

```
# event_manager.gd::_begin_event_internal 의 match 분기에 추가
"combat":
    event_state = {
        "event_id": event_id,
        "kind": "combat",
        "phase": "running",
        "chain_root_id": chain_root,
        "enemy_id": def.enemy_id,
    }
    event_state_changed.emit()
    BattleManager.battle_ended.connect(_on_combat_in_chain_ended, CONNECT_ONE_SHOT)
    RunManager.start_combat({
        "enemy_id": def.enemy_id,
        "body_hp": RunManager.run_data["body_hp"],
        "body_max_hp": RunManager.run_data["body_max_hp"],
        "arm_l": RunManager.get_equipped_arm("L"),
        "arm_r": RunManager.get_equipped_arm("R"),
    })

func _on_combat_in_chain_ended(result: Dictionary) -> void:
    if event_state.is_empty(): return
    event_state["last_combat_result"] = result
    _finalize_event()
```

6-2. on_enter_slot kind="combat" 처리

```
# RunManager._check_on_enter
"combat":
    seen_this_run[slot.id] = true
    run_data["phase"] = "battle_preview"
    run_data["pending_combat"] = {"enemy_id": slot.enemy_id}
    state_changed.emit()
```

또는 EventManager 통과시킬지 결정. 단순 결: 기존 `phase="battle_preview"` 흐름 그대로.

6-3. research 노드 진입 — 이미 GR-2 에서 처리. 재확인.

```
"research":  
    _enter_research()
```

`_enter_research` 가 phase="research" 전이 → research_screen 노출 → leave_research → end_internal_run. 현 흐름 그대로.

검증

- 타일 (X, Y) 진입 → combat 발화 → 전투 → battle_ended → reward
- chain 안 combat — dialogue → combat → dialogue 시퀀스
- research 타일 진입 → phase="research" → leave_research → 회귀

체크 ~15 항목.

§7. Stage GR-6 — UI 그리드 시각 + 입력

작업 범위

`run_ui.gd` : - `_build_map_display` 폐기 → `_build_grid_display` - `_refresh_map_display` 폐기 → `_refresh_grid_display` - `_node_buttons` 폐기 → `_tile_rects` 또는 별 결의 시각

```
const TILE_PX = 40  
const GRID_ORIGIN = Vector2(180, 90)  
  
func _build_grid_display():  
    _grid_root = Control.new()  
    _grid_root.position = GRID_ORIGIN  
    _grid_root.visible = false  
    add_child(_grid_root)  
  
    var rows: int = GameData.WORLD_TERRAIN.size()  
    var cols: int = GameData.WORLD_TERRAIN[0].length()  
    for y in range(rows):  
        for x in range(cols):  
            var t: String = GameData.WORLD_TERRAIN[y][x]  
            var rect = ColorRect.new()  
            rect.position = Vector2(x * TILE_PX, y * TILE_PX)  
            rect.size = Vector2(TILE_PX - 2, TILE_PX - 2)  
            rect.color = _terrain_color(t)  
            _grid_root.add_child(rect)  
            _tile_rects[Vector2i(x, y)] = rect
```

```

# 플레이어 마커
_player_marker = ColorRect.new()
_player_marker.size = Vector2(TILE_PX - 6, TILE_PX - 6)
_player_marker.color = Color(1.0, 0.85, 0.3)
_grid_root.add_child(_player_marker)

func _refresh_grid_display():
    var pos: Vector2i = RunManager.run_data["player_pos"]
    _player_marker.position = Vector2(pos.x * TILE_PX + 3, pos.y * TILE_PX + 3)

    for tile_pos in _tile_rects.keys():
        var rect: ColorRect = _tile_rects[tile_pos]
        var visited: bool = RunManager.run_data["visited_tiles"].get(tile_pos, false)
        rect.modulate = Color(1, 1, 1) if visited else Color(0.5, 0.5, 0.5)

func _unhandled_input(event: InputEvent) -> void:
    if RunManager.run_data.get("phase") != "map": return
    if not (event is InputEventKey) or not event.pressed: return
    match event.keycode:
        KEY_W, KEY_UP:    RunManager.try_move(Vector2i(0, -1))
        KEY_S, KEY_DOWN:  RunManager.try_move(Vector2i(0, 1))
        KEY_A, KEY_LEFT:  RunManager.try_move(Vector2i(-1, 0))
        KEY_D, KEY_RIGHT: RunManager.try_move(Vector2i(1, 0))
        KEY_E:            RunManager.try_explore()
        KEY_R:            RunManager.rest()

```

HUD

- 일자: Day 4 / 10
- 행동: 행동 5 남음
- 회귀 카운트 / 자원 (기준)

검증

- 실 플레이 — main.tscn 인스턴스 + 입력 시물
- 스크린샷 — 그리드 / 플레이어 / HUD / 진입 후 visited 색 변화
- WASD 시물 또는 RunManager 직호출 검증

스크린샷 ~3 컷.

§8. Stage GR-7 — 회귀 테스트 갱신

작업 범위

5 기존 테스트 파일 갱신:

test/test_event_1a.gd — 노드 5/6 → 좌표 변환:

```
# 기존
RunManager.move_to_node(5)
# 신
RunManager.try_move(Vector2i(0, 1)) # SPAWN 에서 노드 5 자리로 가는 dir
# 또는 직접 좌표 세팅 (테스트용)
RunManager.run_data["player_pos"] = Vector2i(...)
```

test/test_event_1a2.gd — 동일 패턴 **test/test_event_1b.gd** — 동일 **test/test_event_2.gd**
— repair chain 시퀀스 좌표 기반 변환 **test/test_save_load.gd** — `current_node_id` → `player_pos`
검증

자가점검

- 모든 테스트 PASS 회복
- 신규 GR 단계 테스트 (GR-1~5) 도 PASS
- 회귀 누적 — 약 130~150 체크

§9. Stage GR-8 — 세이브/로드 호환

작업 범위

`GameManager.save()` / `load_save()` 자체는 dict 통째 직렬화라 변경 0. 단 `Vector2i` 의 `var_to_str` 호환 검증 필요.

```
# 검증 — Vector2i 도 str_to_var 로 round-trip 가능
var v := Vector2i(3, 7)
var s := var_to_str(v)
var v2 = str_to_var(s)
assert(v2 == v)
```

검증 — **test_save_load.gd** 갱신

- save → reset → load
- `player_pos`, `day`, `actions_remaining`, `visited_tiles`, `explored_tiles` 모두 round-trip 일치
- save 시점 정책 — `phase=="map"` 만, 그대로

체크 ~10 항목 추가.

§10. 자가점검 — 단계 진입 전 공통

매 단계 진입 직전 확인:

- [] 본 문서의 해당 §X 작업·검증 부분 변경된 부분 없는지 사용자 합의
- [] 이전 단계 자가점검 모두 통과 상태인지
- [] 선행 spec (그리드 리뉴얼 - 자료구조 · encounter · 트리거 설계.pdf) 의 결정 사항과 충돌 없는지
- [] 원칙 — 삼항 / 자료구조 상향식 / 헤드리스 자동 검증 깨지지 않는지

§11. 흐름 규칙

- 각 단계 시작 전: 본 문서의 해당 §X 작업·검증 부분 사용자와 합의
- 각 단계 종료 시: 자가점검 체크리스트 → 공유 결과물 (콘솔 로그 / diff / 영상) → 사용자 OK 후 다음 단계
- 단계 간 별도 커밋 (브랜치 안). 일괄 PR 후 마스터 merge.
- 막히면 즉시 보고. 명세 변경 동반 시 spec 갱신 → 본 문서 갱신 → 재진입.

§12. 예상 작업 분량

단계	코드 분량	예상 시간
GR-1 자료구조	~80 라인	~3h
GR-2 이동 + on_enter	~120 라인	~5h
GR-3 탐험	~50 라인	~2h
GR-4 휴식 / 일자	~80 라인	~3h
GR-5 combat kind / research	~100 라인	~4h
GR-6 UI 그리드	~250 라인	~6h
GR-7 회귀 테스트	~150 라인 (test 갱신)	~4h
GR-8 save/load	~30 라인	~2h
합	~860 라인	~30h ≈ 5 일

§13. 진입 액션

GO 신호 받으면:

1. `grid-renewal` 브랜치 생성
 2. GR-1 진입 — `game_data.gd` 부터
 3. 매 단계 헤드리스 검증 + 콘솔 로그 / diff 공유
 4. PASS 후 다음 단계
 5. GR-8 까지 완료 + 누적 회귀 PASS → 마스터 merge
-

§14. 후속 (이번 롤아웃 이후)

GR-1~8 종료 후 자연스러운 다음 차로: - 콘텐츠 작성 — `TILE_ENCOUNTERS` 의 본격 박힘 (1구역 / 2구역 narrative) - `REST_EVENTS` 풀 확장 - 알 발견 / 자폭 / 진엔딩 시스템 - 식량 / 시간 / 계절 등 추가 자원 - 비주얼 — 수채화 / SF 결 결정 + 자산